Wordle Solver Project – ALGO3 ISIL

Your Name / Group Members

December 19, 2025

Part 3 – Analysis and Documentation

1 Strategy Description

1.1 Manual Game Strategy (Part 1)

- **Target word selection:** Randomly chosen from `words.txt`.
- **Guessing process:** Player inputs a 5-letter word in each attempt.
- **Feedback:**
  - `[G]`: Correct letter, correct position.
  - `[Y]`: Correct letter, wrong position.
  - `[ ]`: Letter not in the word.
- **Goal:** Help the player reduce possible words in subsequent guesses.

1.2 Automated Solver Strategy (Part 2)

- **Candidate initialization:** All words from the dictionary are stored as candidates.
- **Guess selection:** First candidate word is chosen for the current guess.
- **Feedback calculation (`result`):**
  - `G` = correct letter and position.
  - `Y` = correct letter, wrong position.
  - `B` = letter not in the word.
- **Candidate filtering:** Remove all words inconsistent with previous feedback.
- **Iteration:** Repeat until the word is found or maximum attempts are reached.
- **Effectiveness:** Quickly reduces possible words, increasing chances of finding the target word in minimal guesses.

2 Data Structure Justification

- **Dictionary array** `char words[MAX_WORDS][WORD_LENGTH+1]`: Stores all dictionary words with fast random access.
- **Candidate array** `char candidates[MAX_WORDS][WORD_LENGTH+1]`: Stores currently possible words after each guess.
- **Alternative considered:** Linked list, less efficient for random access.
- **Support for strategy:** Arrays allow fast filtering and next-guess selection.

3 Complexity Analysis

3.1 Time Complexity

- Loading words from file: $O(n)$ where n = number of words.
- Filtering candidates after each guess: $O(m \times WORD\_LENGTH)$ where m = current number of candidates.
- Total complexity: $O(n \times MAX\_ATTEMPTS \times WORD\_LENGTH)$.

3.2 Space Complexity

- Dictionary storage: $O(n \times WORD\_LENGTH)$
- Candidate storage: $O(n \times WORD\_LENGTH)$
- Temporary arrays (`result`, `temp_feedback`): $O(WORD\_LENGTH)$

4 Code Documentation

4.1 `check_letters`

```
1 void check_letters(const char *guess, const char *target, char *result
    );
```

- **Purpose:** Compare guess with target and compute feedback.
- **Parameters:**
  - guess: Current guessed word
  - target: Secret word
  - result: Stores feedback ('G','Y','B')

4.2 `is_valid`

```
1 int is_valid(const char *candidate, const char *guess, const char *
    result);
```

- **Purpose:** Check if a candidate word is consistent with previous guess feedback.
- **Returns:** 1 if valid, 0 otherwise.

4.3 `main`

- Loads words from file.
- Selects a random target word.
- Initializes candidate list.
- Iteratively guesses, computes feedback, and filters candidates.

5 Example Output

5.1 Manual Game (Part 1)

```
Welcome to Wordle!
You have 6 attempts to guess the 5-letter word.
Attempt 1: Enter word: PLACE
[ G ][   ][   ][ Y ][   ]  PLACE
Attempt 2: Enter word: PLANT
[ G ][ G ][   ][   ][   ]  PLANT
Attempt 3: Enter word: PLATE
[ G ][ G ][ G ][ G ][ G ]  PLATE
Congratulations! You guessed the word!
```

5.2 Automated Solver (Part 2)

```
Wordle Solver
Attempt 1: PLACE -> GBBYB
Attempt 2: PLANT -> GGBGB
Attempt 3: PLATE -> GGGGG
Word correct!
```